Information-cell theory - A breakthrough in symbolic AI

Agenda

- Introduction to InfoCell (45 min)
- Live demo (5 min)
- Q&A (10 min)

The master plan for a proto AGI

01

Using C++ to create infocells

Infocell objects are created with standard C++ methods so no parser is needed

02

Programming language without a syntax

We can create a regular looking programming language with infocells by directly creating AST nodes

03

Cells can be activated

We can create a runnable instruction-set with infocells, so compiling an infocell program to an info-cell instruction-set will be still infocell

04

Extending the language

We can introduce description-based grammar to express what an instruction or composed instructions (methods) does.

05

Using the new grammar as a database

We use the description grammars as a database of action results

06

Creating prompts with the new grammar

We can describe an outcome with description grammar, so we can define a goal

07

We can directly relate two prompts

As we have a database of action results, we can directly synthesize instructions that convert one prompt to another one. This is similar to the back-propagation. This is the end-goal to solve the ARC-AGI challenges.

01 What is an InfoCell

Visualizing cells

Core idea

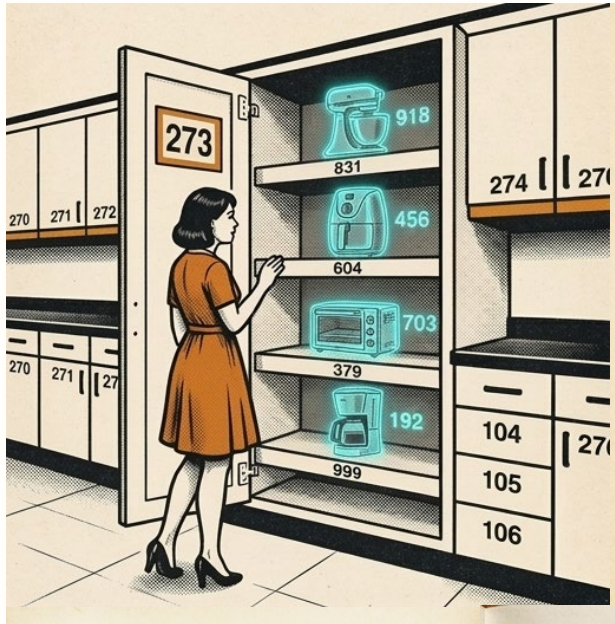
An info cell is essentially an object just like an object from a regular OOP world. It has a unique address and can contains key-value pair of cell addresses.

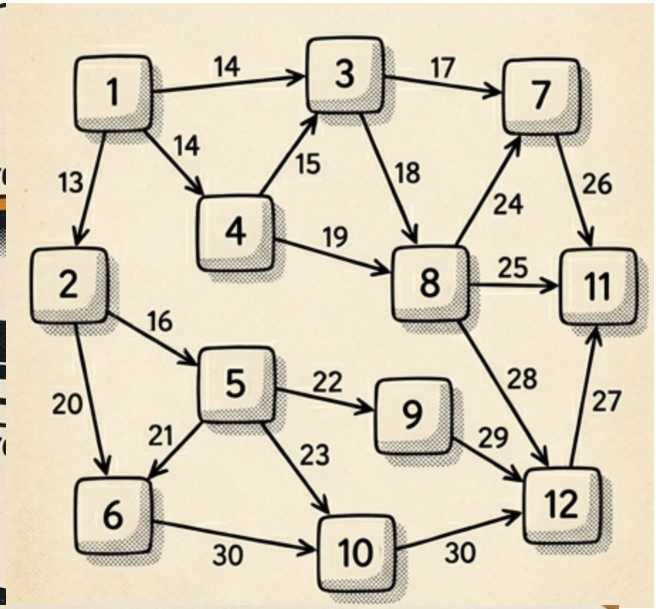
Duality

These cells are forming a graph, where the edges are actually infocell references.

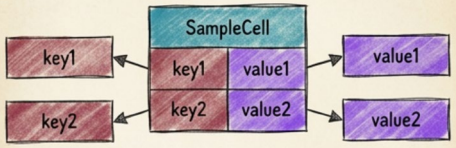
Illustration

These 5 image try to explain the same concept. Every cell represent something.





Here a simple cell called SampleCell it contains two slots.
In the first slot the slotRole (key) is key1 and the slotValue (value) is value1.
In the second slot the slotRole (key) is key2 and the slotValue (value) is value2.
Here key1, key2, value1 and value2 are empty cells.




```
Object key1(kb, kb.ids.emptyObject);
Object key2(kb, kb.ids.emptyObject);
Object value1(kb, kb.ids.emptyObject);
Object value2(kb, kb.ids.emptyObject);

Object sampleCell(kb, kb.std.Index);
sampleCell.set(key1, value1);
sampleCell.set(key2, value2);
```

```
class Object
{
public:
    std::map<Object*, Object*> m_slots;
};
```

02 Programming language without a syntax

Creating a language inside a language

Core idea

The C++ methods are creating directly language AST node infocells. These AST data cells can be compiled to active cells (called OP cells). The compiler can also be an infocell program.

Developing experience

We have IDE, parser, syntax highlighting for free. “Not great, not terrible”, kind of a hack, but works! The compiler has some basic checks in it.

Language features

We have templates (with implicit template instantiation), subtypes, Rust style enums, structs, methods, Rust traits (almost done), do … while loop, while loop, if … then, easy to extend.

```
// struct Frame
auto& frameStruct
    = arcScope.add<Struct>("Frame")
          .subTypes(
              parameter("tableType", tt_("std::Map", "keyType", _(std.Number), "valueType", tt_("std::Map", "keyTy
          .members(
              member("width", _(std.Number)),
              member("height", _(std.Number)),
              member("grid", _(std.Grid)),
              member("edgeNodes", tt_("std::List", "valueType", "ShapeEdgeNode")),
              member("shapePixels", st_("tableType")),
              member("upLeftPoint", "ShapePoint"),
              member("upRightPoint", "ShapePoint"),
              member("downLeftPoint", "ShapePoint"),
              member("downRightPoint", "ShapePoint"),
              member("shapes", tt_("std::List", "valueType", "Shape")),
              member("shapeMap", tt_("std::Map", "keyType", _(std.Number), "valueType", "Shape")),
              member("inputPixels", tt_("std::Set", "valueType", _(std.Pixel))));

// Frame::Frame
frameStruct.addMethod("constructor")
    .parameters(
        parameter("grid", _(std.Grid)))
    .instructions(
        m_("grid")        = p_("grid"),
        m_("width")       = p_("grid") / "width",
        m_("height")      = p_("grid") / "height",
        m_("shapes")      = new_(tt_("std::List", "valueType", "Shape"), "constructor"),
        m_("shapePixels") = new_(st_("tableType"), "constructor"),
        m_("shapeMap")    = new_(tt_("std::Map", "keyType", _(std.Number), "valueType", "Shape"), "constructor"),
        m_("inputPixels") = new_(tt_("std::Set", "valueType", _(std.Pixel)), "constructor"),
        self()("processInputPixels"));

// Frame::processInputPixels
frameStruct.addMethod("processInputPixels")
    .instructions(
        var_("pixels") = m_("grid") / "pixels",
        if_(missing(*var_("pixels"), "first"))
            .then_(return_()),
        var_("pixel") = *var_("pixels") / "first",
        while_(true_())
            .do_(block(
                m_("inputPixels")("add")("value", *var_("pixel") / "value"),
              if_(missing(*var_("pixel"), "next"))
                  .then_(return_()),
              var_("pixel") = *var_("pixel") / "next")));
```

03 Cells can be activated

The activation

Core idea

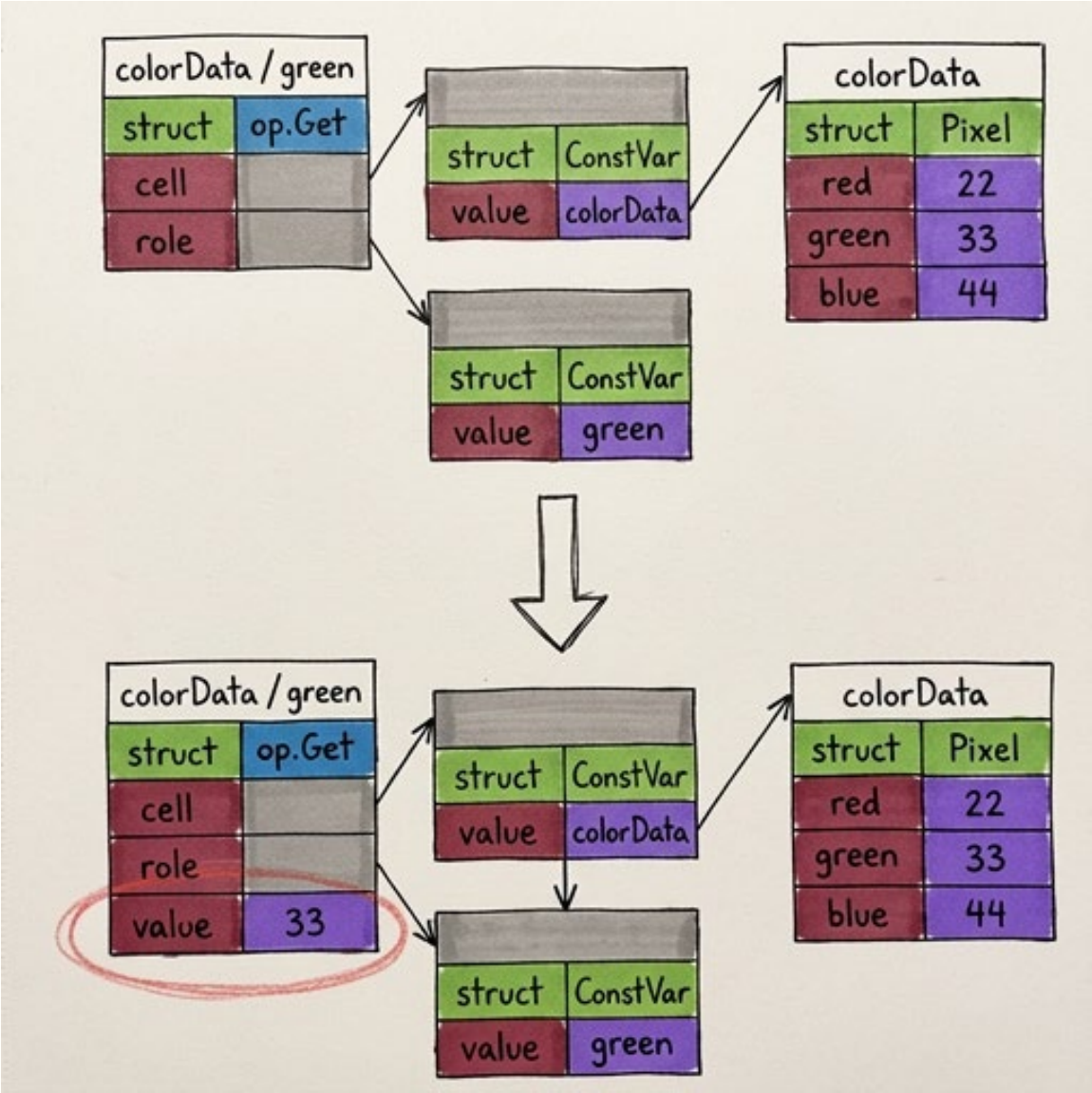
The activated cell itself will store the result. It will change itself.

Wrapping the values

Without wrapping (using an extra ConstVar node) we are unable to inspect active nodes.

Composition

We can put active active cell inside an active cell. Activating datacell (here ConstVar) is fine (nothing happen).



04 Extending the language

Indicative mood (description)

Core idea

Indicative mood is not an instruction. It is a description. You don’t have to do anything. Can not be compiled to an instruction set. It is not executable. It is an observation.

Imperative programming languages

The current imperative programming languages are actually using parametric imperative mood grammars. Instructions. There are declerative programming languages (Haskel, Prolog, …), but they don’t use instructions. We need both world.

The description segment

Core idea

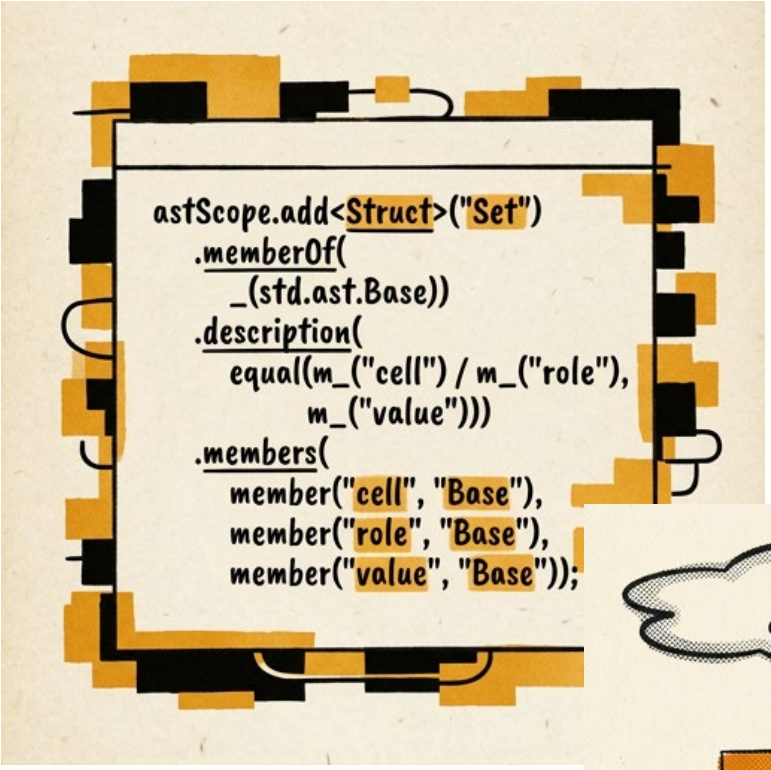
We can add a description segment to an AST node. It is looks like an equation.

Expressing observation

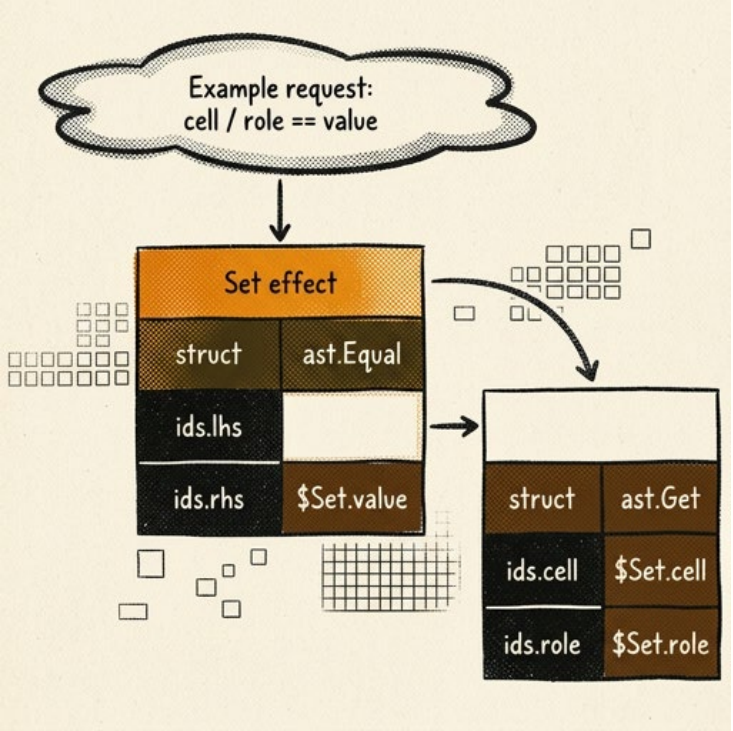
We can observe the active cell SET. After activating it, a cell will be changed. We can observe the changed cell with a GET cell. So we can measure the changed cell with the GET cell.

Knowledge base

There is a striking resemblance to Unit Tests. We write unit tests to execute them. But those are actually observations also, so they can be used to build a knowledge base.



```
astScope.add<Struct>("Set")
    .memberOf(
        _(std.ast.Base))
    .description(
        equal(m_("cell") / m_("role"),
              m_("value")))
    .members(
        member("cell", "Base"),
        member("role", "Base"),
        member("value", "Base"));
```



05 Using the new grammar as a database

Storing observations

Core idea

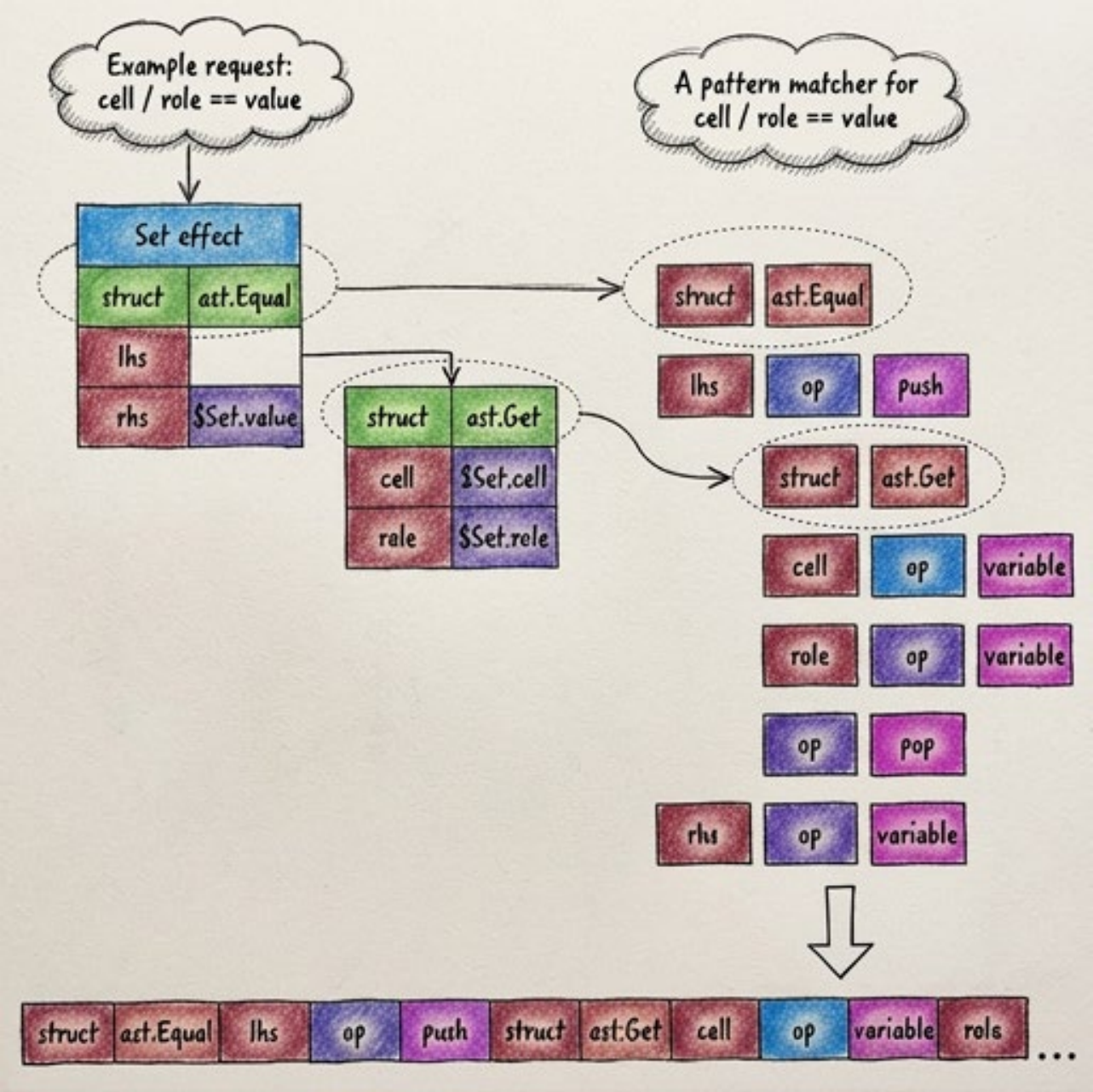
We can store the effect an activation of an active tool to later look up what cell / method can cause such an effect. We can do a reverse lookup (or reverse parsing maybe?).

The storage method

First we serializing the parametrized observation then we can put it into a trie datastructure.

Variables

The matcher contains no knowledge about how a variable will be used later. We have to connect this matcher to a synthetizer that actually synthetize code from a match.



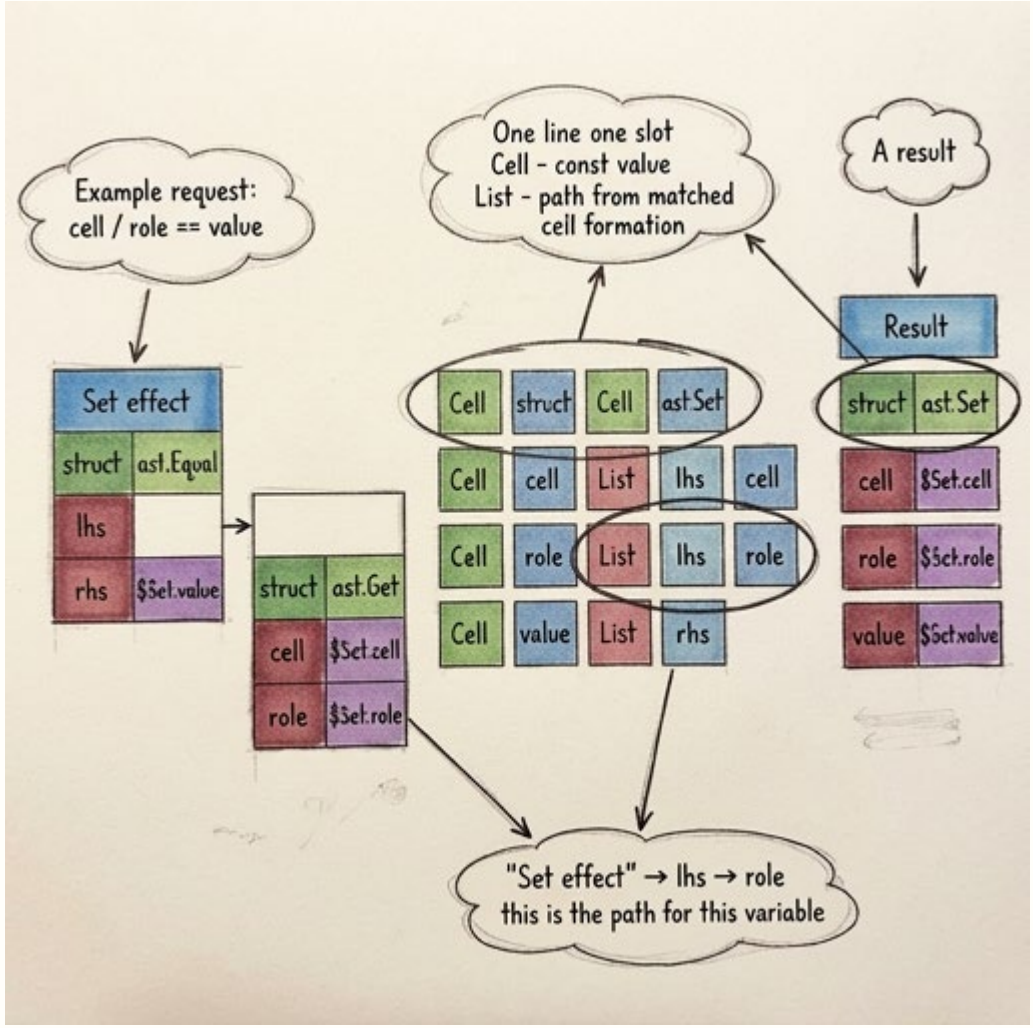
Synthetizing code

Core idea

The matcher and the code synthetizer are two separate cell formation. First, we match with a matcher, then we cherry-pick values from the matched formation.

Knowledge base

There is a striking resemblance to unit tests. We write unit tests to execute them, but they are essentially observations as well, so they can be used to build a knowledge base. We solve the unit tests, and the result is code that fulfills them.



06 Creating prompts with the new grammar

Prompts

Core idea

We can use a standard AST block for prompt. An AST block usually used for function body, the { … } part. We can create local variables here, so it won't affect the main database.

Learning

To modify the main database, we have to reference an already existing object. To prevent accidental modification we need const objects in the database.

Query

For query the main database we have to create an empty variable in the prompt and describing that variable. The symbolic transformer will synthetize

```
// currentTheme.get(std.Color).get(green) == 5
RequestHelper(brain::Brain& kb) :
    AstHelper(kb)
{
    Var& currentTheme = var_("currentTheme");
    Base& ast = equal(_(currentTheme)/_(std.Color)/_(ids.green), _(5_));
    value = &ast;
}
```

Mathematical functions as prompts

Core idea

We can “observe” the result of an addition (+) with a subtraction (-) mathematical function.

Relating two number

We can use this observation to relate two number together. How we can go from 4 to get 2? We can find the subtract math function and we can synthetize an observed relation function here.

Under construction

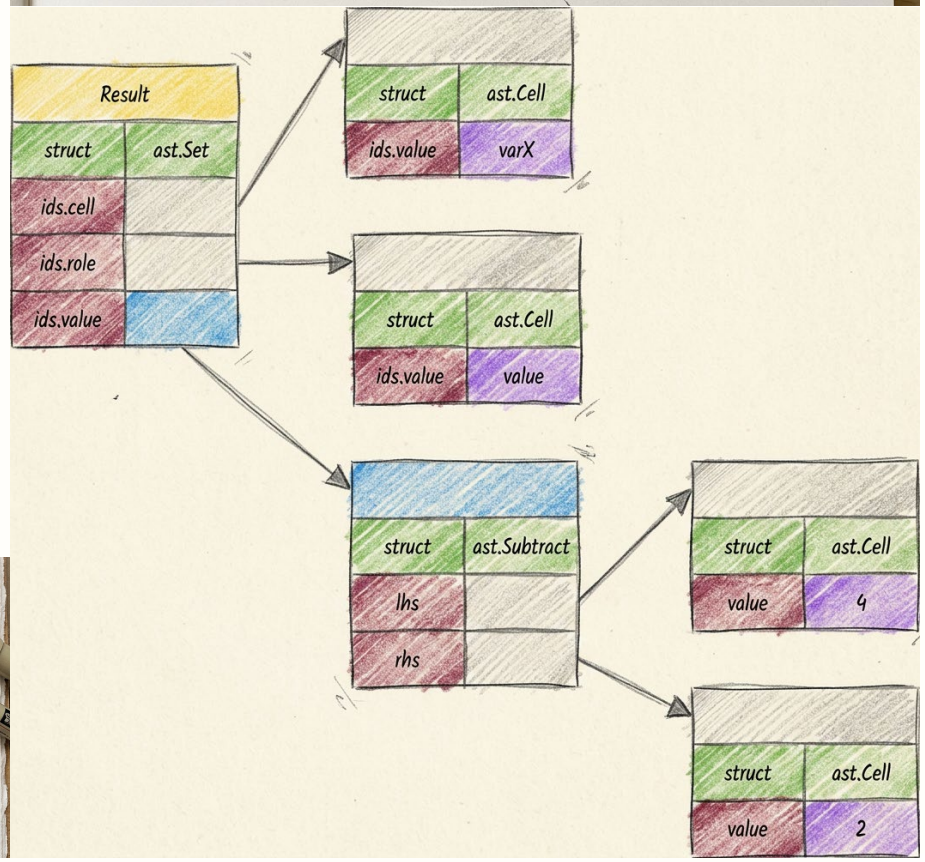
Although this example is working under unit tests, the 2 + x =4 case currently doesn’t work.

```
RequestHelper(brain::Brain& kb) :
    AstHelper(kb)
{
    Var& x = var_("x");

    // test x + 2 = 4 => equal(add(get(x, value), 2)), 4)
    Base& ast = equal(add(_(x) / _(kb.ids.value), _(_2_)), _(_4_));
    varX    = &x;
    request = &ast;
}
```

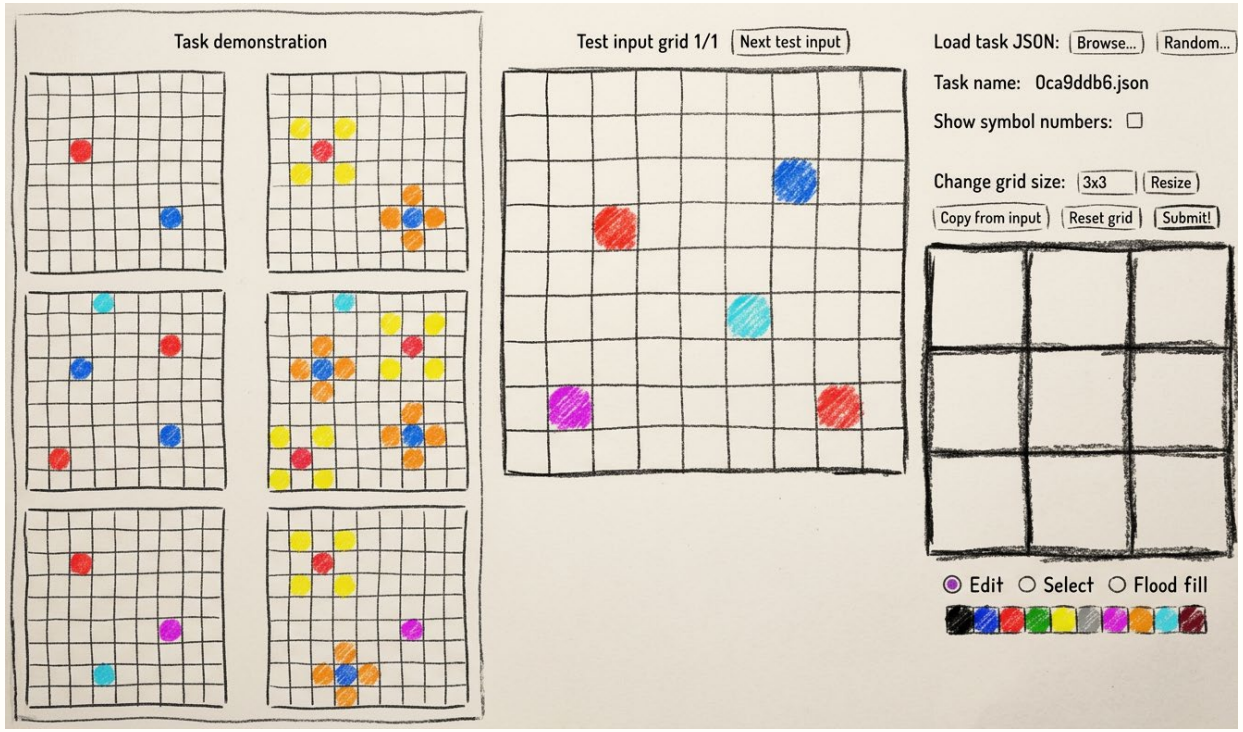
```
astScope.add<Struct>("Subtract")
    .memberOf(
        _(std.ast.Base))
    .description(
        equal(add(return_(), m_("rhs")), m_("lhs")),
        equal(add(m_("rhs"), return_()), m_("lhs")),
        equal(m_("lhs"), add(return_(), m_("rhs"))),
        return_(subtract(m_("lhs"), m_("rhs"))))
    .members(
        member("lhs", "Base"),
        member("rhs", "Base"));
```

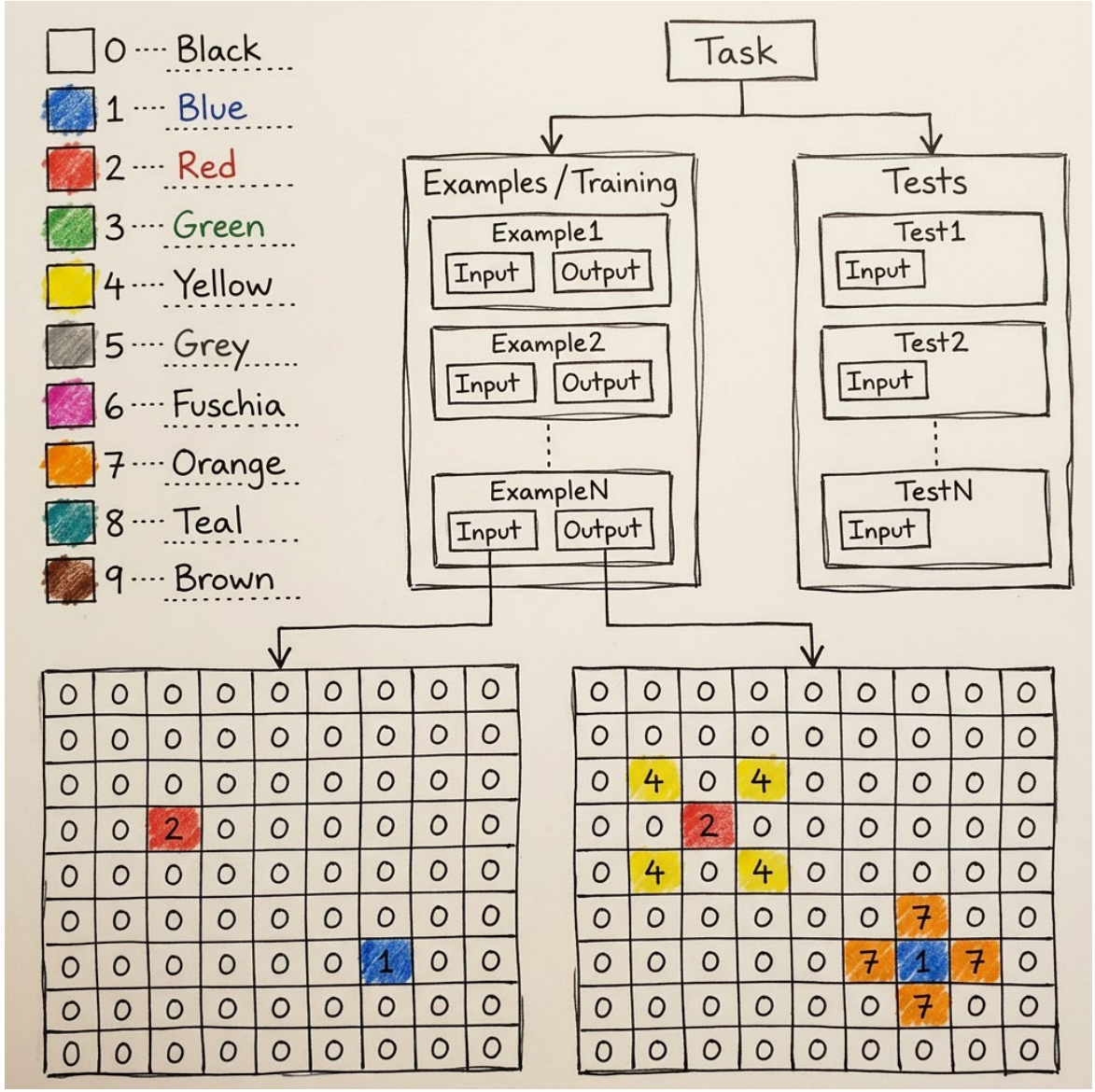
Core logic



07 We can directly relate two prompts

An ARC-AGI task

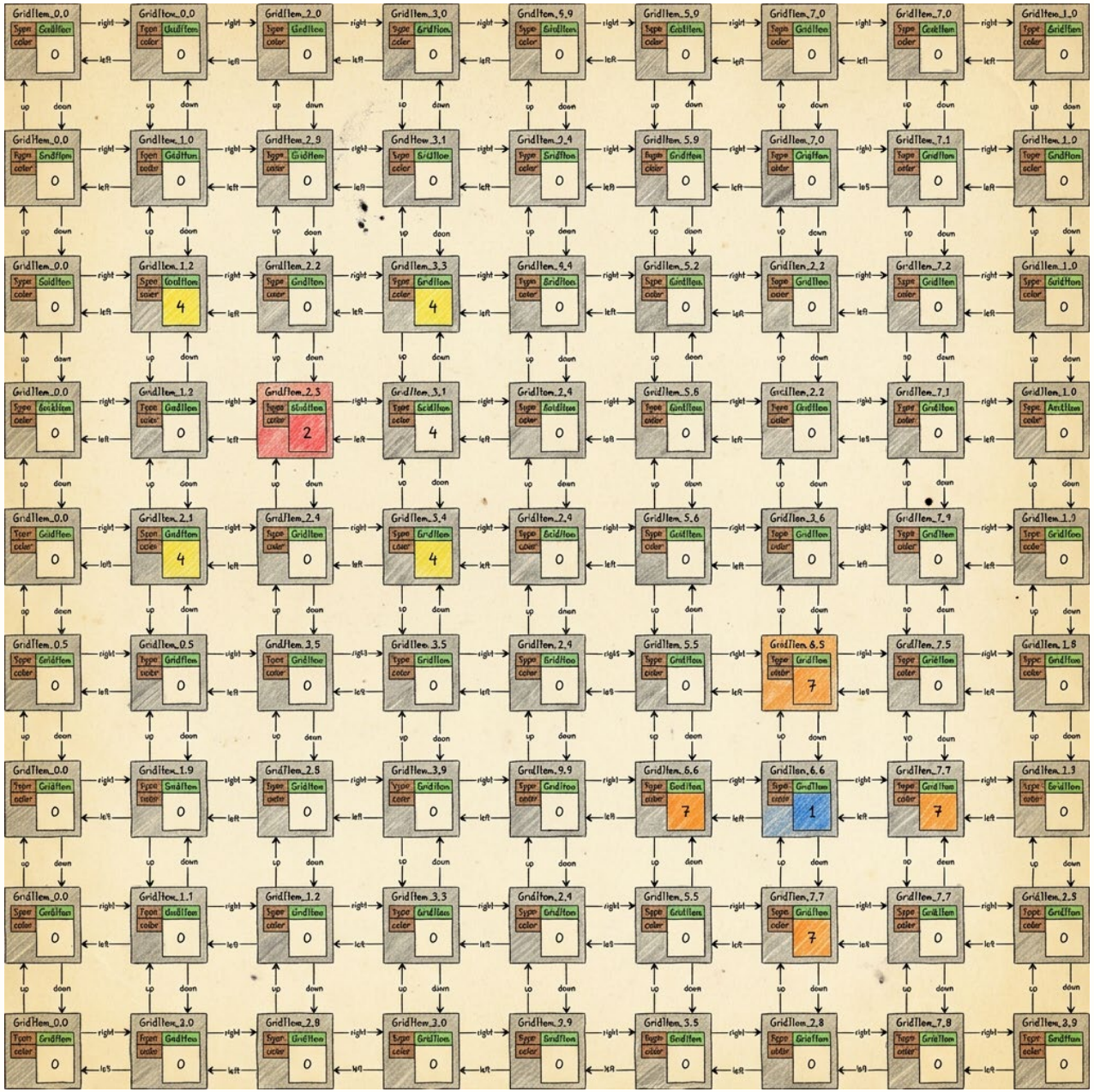




Representing the grid with infocells

Core idea

We can create a grid of cells to represent a screen. The cells are connected with { left, right, up, down } connections.



Representing the grid with infocells

Edge detection

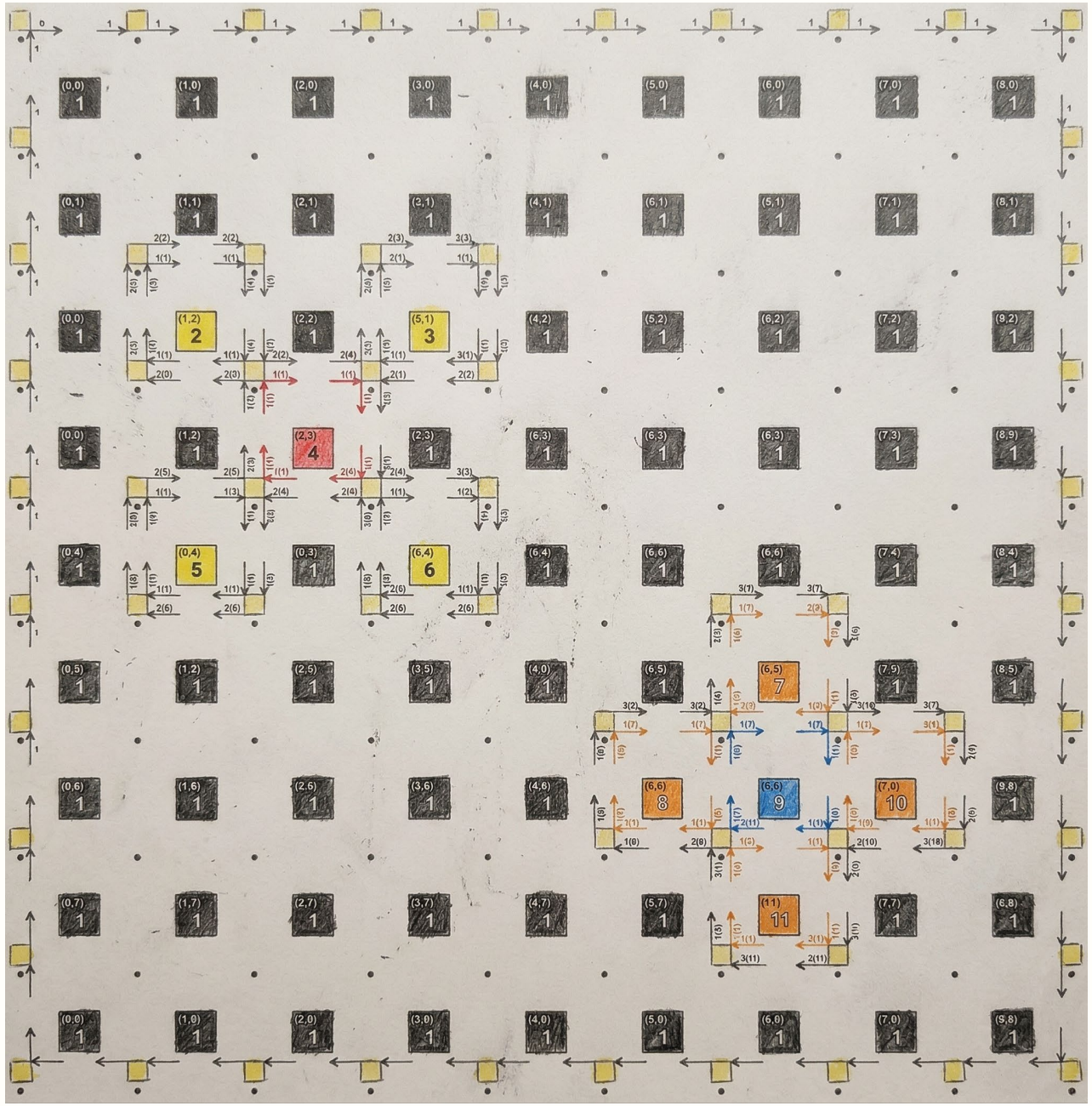
We can separate the same-colored blobs by detecting edges. We can store the edges by a series of (parametric) unit vectors.

Legend

Here the numbers are object numbers not color code numbers.

Storage and recognition

We can use the same pattern matcher algorithm and storage method as we do with AST nodes. Here we store parametric unit (edge) vectors.



We can directly relate one prompt to the other.

Core idea

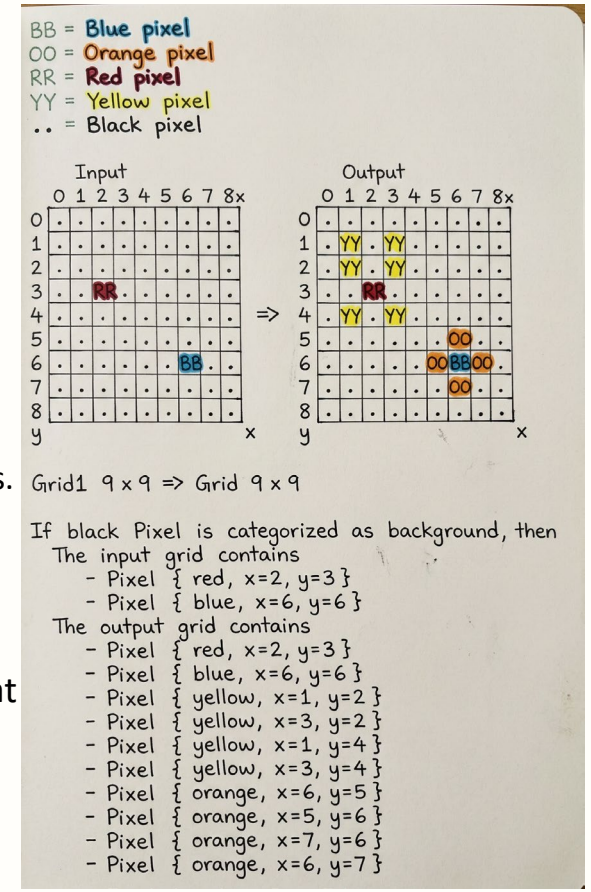
We can create a description from a picture with the help of observer functions. We can create as many interpretations as we want. We create a description for the input and the output image. Then we can synthetize as many conversion code as possible.

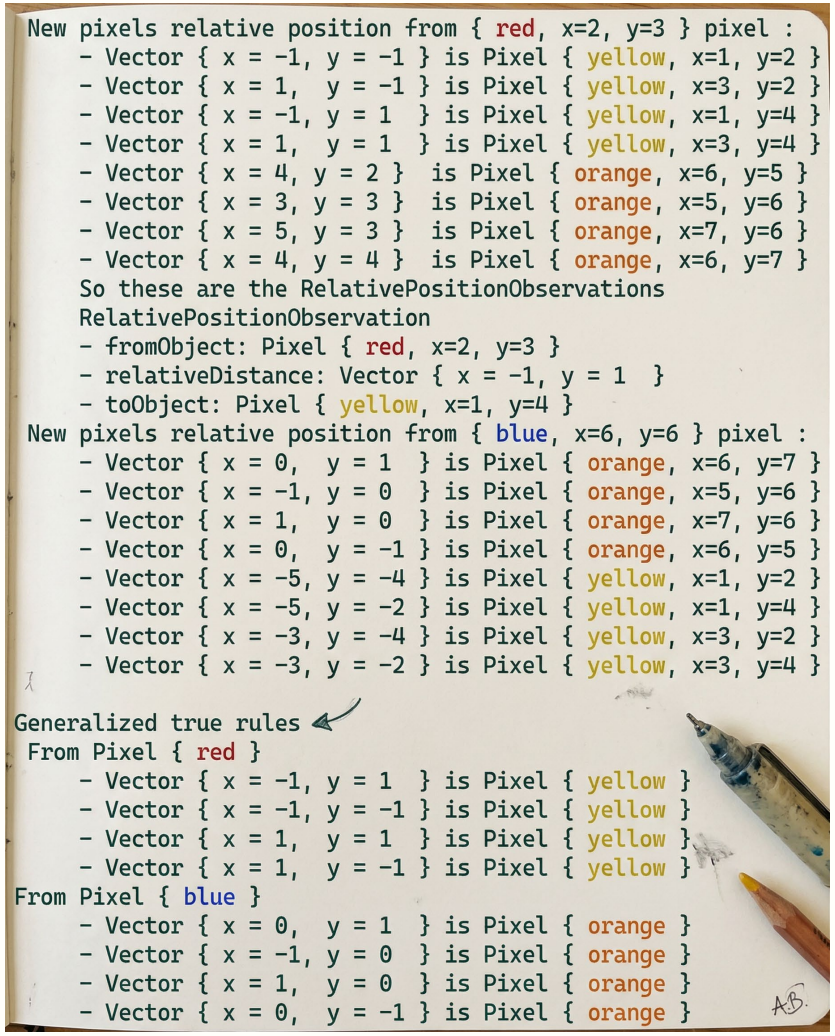
Filtering

For every pair we create a candidates of transformation rules. We keep the one that is the same for every pair.

Synthetizing the winning transformation

The last step is to synthetize the actual active nodes that do the transformation.





More info & links

Author

Péter Németh, software architect at Budapest, Hungary

Repositories

- My InfoCell paper & POC: https://github.com/hun-nemethpeter/InfoCell
- Original ARC-AGI by François Chollet: https://github.com/fchollet/ARC-AGI

Links

- Original ARC paper (On the measure of intelligence by François Chollet): https://arxiv.org/abs/1911.01547
- ARC-PRIZE challenge main website: https://arcprize.org/
- Kaggle is hosting the challenge: https://www.kaggle.com/competitions/arc-prize-2026-arc-agi-2/

Demo

Q & A